

Labolatorium nr 8

Temat: Tworzenie agentów AI MCP

1 Stworzenie prostego serwera MCP w C#

1.1 Architektura

Serwer MCP zbudowany jest na bazie:

- Framework: **.NET 9.0 Console Application**
- Komunikacja: **JSON over stdin/stdout** oraz **ASP.NET Core Web API (HTTP)**
- Port: **5000** (lokalna komunikacja)

1.2 Inicjalizacja projektu

```
PS D:\stud\sem 5\Desktop\z8> dotnet new console -n MCPServer
```

```
PS D:\stud\sem 5\Desktop\z8\MCPServer> dotnet build
```

1.3 Program.cs - Definicja endpointów

Serwer obsługuje następujące komendy:

```
// Endpoint zdrowia
app.MapGet("/", () => new {
    status = "ok",
    message = "MCP Server running",
    version = "3.0.0"
});

// Endpoint MCP
app.MapPost("/mcp", async (HttpContext context) =>
{
    using var reader = new StreamReader(context.Request.
        Body);
    var line = await reader.ReadToEndAsync();

    var json = JsonDocument.Parse(line);
    var root = json.RootElement;
    string command = root.GetProperty("command").
        GetString() ?? "";

    string response = command switch
    {
        "init" => JsonSerializer.Serialize(new
        {
            status = "initialized",
            name = "MCPServerWithDatabase",
            version = "3.0.0"
        }),
    }
```

```
        "list_tables" => JsonSerializer.Serialize(  
            new  
            {  
                status = "ok",  
                data = dbServer.ListTables()  
            },  
  
        "query_database" => HandleQueryDatabase(root  
            , dbServer),  
        "insert_data" => HandleInsertData(root,  
            dbServer),  
        "update_data" => HandleUpdateData(root,  
            dbServer),  
        "delete_data" => HandleDeleteData(root,  
            dbServer),  
  
        _ => JsonSerializer.Serialize(new { error =  
            "Unknown command" })  
    };  
  
    await context.Response.WriteAsync(response);  
});  
  
app.Run();
```

1.4 Obsługiwane komendy

Komenda	Opis
init	Inicjalizacja serwera
list_tables	Wylistowanie tabel w bazie
query_database	SELECT z bazy danych
insert_data	INSERT do bazy
update_data	UPDATE rekordu
delete_data	DELETE rekordu

1.5 Test lokalny

```
PS D:\stud\sem 5\Desktop\z8\MCPServer> dotnet run
```

```
Building...
```

```
Starting...
```

```
info: Microsoft.Hosting.Lifetime[14]
```

```
Now listening on: http://localhost:5000
```

```
// Test endpointu
```

```
curl http://localhost:5000/
```

```
> {"status":"ok","message":"MCP Server running","version":"3.0.0"}
```

1.6 Wnioski

Serwer MCP został pomyślnie stworzony z pełną obsługą protokołu JSON. Architektura umożliwia łatwą rozbudowę o nowe komendy poprzez dodanie nowych gałęzi w switch'u.

2 Klient MCP – wywołanie narzędzia MCP przez LLM z kodu C#

2.1 Cel

Stworzenie klienta, który komunikuje się z serwerem MCP i integruje się z API Groq LLM do przetwarzania naturalnego języka.

2.2 Architektura

```
[User Query]
  (|)
[Groq LLM]
  (|) (HTTP)
[MCPClient]
  (|) (HTTP REST)
[MCPServer na Azure]
  (|) (SQLite)
[Database]
```

2.3 MCPIntegrator_Azure.cs - Klasa integracyjna

```
public class MCPIntegratorAzure
{
    private readonly string _apiKey;
    private readonly HttpClient _httpClient;
    private const string AZURE_SERVER_URL =
        "https://mcp-server-app-pms.azurewebsites.
        net";
    private const string GROQ_API_URL =
        "https://api.groq.com/openai/v1/chat/
        completions";

    public MCPIntegratorAzure(string apiKey)
    {
        _apiKey = apiKey;
        _httpClient = new HttpClient();
    }

    public string SendToMCPServer(string jsonCommand)
    {

```

```
        try
        {
            var jsonContent = new StringContent(
                jsonCommand,
                Encoding.UTF8,
                "application/json"
            );

            var response = _httpClient.PostAsync(
                (
                    $"{AZURE_SERVER_URL}/mcp",
                    jsonContent
                ).Result;

            string content = response.Content.
                ReadAsStringAsync().Result;
            return content;
        }
        catch (Exception ex)
        {
            return $"ERROR: {ex.Message}";
        }
    }

    public async Task<string> CallGroqLLM(string
        userMessage)
    {
        var request = new
        {
            model = "meta-llama/llama-4-scout-17
                b-16e-instruct",
            messages = new[]
            {
                new { role = "system",
                    content = "Jesteś asystentem
                        mogącym wywoływać narzę
                        dzia MCP." },
                new { role = "user", content
                    = userMessage }
            },
            temperature = 0.7,
            max_tokens = 500
        };

        var jsonContent = new StringContent(
            JsonSerializer.Serialize(request),
            Encoding.UTF8,
            "application/json"
        );
```

```
        var requestMsg = new HttpRequestMessage(
            HttpMethod.Post, GROQ_API_URL)
        {
            Content = jsonContent,
            Headers = { { "Authorization", $"
                Bearer {_apiKey}" } }
        };

        var response = await _httpClient.SendAsync(
            requestMsg);
        var responseContent = await response.Content
            .ReadAsStringAsync();

        var jsonDoc = JsonDocument.Parse(
            responseContent);
        return jsonDoc.RootElement
            .GetProperty("choices")[0]
            .GetProperty("message")
            .GetProperty("content")
            .GetString() ?? "";
    }
}
```

2.4 Program.cs - Sesja interaktywna

```
string groqApiKey = Environment.GetEnvironmentVariable("
    GROQ_API_KEY") ?? "";

var integrator = new MCPIntegratorAzure(groqApiKey);

try
{
    integrator.StartMCPServer();
    await integrator.InteractiveSession();
}
finally
{
    integrator.StopMCPServer();
}
```

2.5 Konfiguracja Groq API

1. Rejestracja: <https://console.groq.com>
2. Pobranie API Key: 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
3. Ustawienie zmiennej:
\$env:GROQ_API_KEY = "gsk_..."

4. Test:
dotnet run

2.6 Model LLM

Właściwość	Wartość
Model	meta-llama/llama-4-scout-17b-16e-instruct
Provider	Groq API
Temperatura	0.7
Max tokens	500
Status	Aktywny (nie wycofany)

2.7 Wnioski

MCPClient pomyślnie integruje Groq LLM z serwerem MCP, umożliwiając przetwarzanie zapytań użytkownika w naturalnym języku i komunikację z bazą danych.

3 Rozszerzenie serwera MCP – dostęp do bazy danych

3.1 Cel

Integracja SQLite z serwerem MCP, umożliwiająca pełne operacje CRUD na bazie danych.

3.2 DatabaseServer.cs - Zarządzanie bazą danych

```
public class DatabaseServer
{
    private SqlConnection _connection;

    public DatabaseServer(string dbPath)
    {
        _connection = new SqlConnection($"Data
            Source={dbPath}");
        _connection.Open();
        InitializeTables();
        SeedData();
    }

    private void InitializeTables()
    {
        // Tabela Users
        var userCmd = _connection.CreateCommand();
```

```
        userCmd.CommandText = @"
            CREATE TABLE IF NOT EXISTS Users (
                Id INTEGER PRIMARY KEY,
                Name TEXT,
                Email TEXT,
                Created DATETIME
            )";
        userCmd.ExecuteNonQuery();

        // Tabela Products
        var productCmd = _connection.CreateCommand();
        ;
        productCmd.CommandText = @"
            CREATE TABLE IF NOT EXISTS Products
            (
                Id INTEGER PRIMARY KEY,
                Name TEXT,
                Price REAL,
                Stock INTEGER
            )";
        productCmd.ExecuteNonQuery();
    }

    public string QueryDatabase(string table, string
        where = "")
    {
        var cmd = _connection.CreateCommand();
        cmd.CommandText = $"SELECT * FROM {table}";
        if (!string.IsNullOrEmpty(where))
            cmd.CommandText += $" WHERE {where}";

        var reader = cmd.ExecuteReader();
        return ReadResults(reader);
    }

    public string InsertData(string table, Dictionary<
        string, string> values)
    {
        var cmd = _connection.CreateCommand();
        var columns = string.Join(", ", values.Keys)
        ;
        var placeholders = string.Join(", ",
            values.Keys.Select(k => $"@{k}"));

        cmd.CommandText =
            $"INSERT INTO {table} ({columns})
            VALUES ({placeholders})";

        foreach (var kvp in values)
```



```
        {
            cmd.Parameters.AddWithValue($"@{kvp.Key}", kvp.Value);
        }

        cmd.ExecuteNonQuery();
        return "SUCCESS: Wstawiono rekord";
    }

    public string UpdateData(string table,
        Dictionary<string, string> setValues, string
        where)
    {
        var cmd = _connection.CreateCommand();
        var setClause = string.Join(", ",
            setValues.Keys.Select(k => $"{k} = @
            {k}"));

        cmd.CommandText = $"UPDATE {table} SET {
            setClause} WHERE {where}";

        foreach (var kvp in setValues)
        {
            cmd.Parameters.AddWithValue($"@{kvp.Key}", kvp.Value);
        }

        cmd.ExecuteNonQuery();
        return "SUCCESS: Zaktualizowano rekord";
    }

    public string DeleteData(string table, string where)
    {
        var cmd = _connection.CreateCommand();
        cmd.CommandText = $"DELETE FROM {table}
            WHERE {where}";
        cmd.ExecuteNonQuery();
        return "SUCCESS: Usunieto rekord";
    }

    public string ListTables()
    {
        return "Tabele: Users, Products";
    }
}
```

3.3 Sample Data - Seeding

Tabela Users:

Id	Name	Email	Created
1	Jan Kowalski	jan@example.com	2025-12-19
2	Maria Nowak	maria@example.com	2025-12-19
3	Piotr Lewandowski	piotr@example.com	2025-12-19

Tabela Products:

Id	Name	Price	Stock
1	Laptop	3499.99	5
2	Mysz	49.99	50
3	Klawiatura	199.99	20

3.4 Testowanie operacji CRUD

3.4.1 Test 1: List Tables

Request: {"command": "list_tables"}
Response: {"status": "ok", "data": "Tabele: Users, Products"}
Status: TAK (PASS)

3.4.2 Test 2: Query Users

Request: {"command": "query_database", "query": {"table": "Users"}}
Response: {"status": "ok", "data": "3 records returned"}
Status: TAK (PASS)

3.4.3 Test 3: Insert Data

Request: {"command": "insert_data",
 "insert": {"table": "Users",
 "values": {"Name": "Anna", "Email": "anna@example.com"}}}
Response: {"status": "ok", "data": "SUCCESS: Wstawiono 1 rekord"}
Status: TAK (PASS)

3.4.4 Test 4: Update Data

Request: {"command": "update_data",
 "update": {"table": "Products",
 "set": {"Price": "3499.99"},
 "where": "Id=1"}}
Response: {"status": "ok", "data": "SUCCESS: Zaktualizowano 1 rekord"}
Status: TAK (PASS)

3.4.5 Test 5: Delete Data

Request: {"command": "delete_data",
 "delete": {"table": "Users", "where": "Id=4"}}
Response: {"status": "ok", "data": "SUCCESS: Usunięto 1 rekord"}
Status: TAK (PASS)

4 Deployment na Azure

4.1 Cel

Wdrożenie aplikacji MCPServer na platformę Azure Web App z możliwością komunikacji przez HTTP.

4.2 Przygotowanie infrastruktury

4.2.1 Zasoby Azure

Zasób	Konfiguracja
Resource Group	mcp-resource-group
Region	West Europe
App Service Plan	B1 (Basic)
Free Tier	Do 18.01.2026
Runtime	.NET 9.0 Core
Web App	mcp-server-app-pms

4.2.2 Azure CLI - Tworzenie zasobów

```
// 1. Zalogowanie
az login --tenant 3f7764cf-7717-4f10-a7d2-f5d8cc83177d

// 2. Grupa zasobów
az group create --name mcp-resource-group --location westeurope

// 3. Plan usług
az appservice plan create --name mcp-app-plan \
  --resource-group mcp-resource-group --sku B1 --is-linux

// 4. Aplikacja web
az webapp create --resource-group mcp-resource-group \
  --plan mcp-app-plan --name mcp-server-app-pms \
  --runtime DOTNETCORE:9.0
```

4.3 Publikacja i wdrażanie

4.3.1 Build Release

```
PS D:\stud\sem 5\Desktop\z8\MCPServer> dotnet publish -c Release -o ./publish
```

Output:

- mcp_database.db
- MCPServer.dll
- appsettings.json

- Entire .NET runtime

Size: 17,919,408 bytes (app.zip)

4.3.2 Deployment do Azure

```
// Tworzenie ZIP  
Compress-Archive -Path ./publish/* -DestinationPath app.zip -Force
```

```
// Wdrożenie  
az webapp deployment source config-zip \  
  --resource-group mcp-resource-group \  
  --name mcp-server-app-pms \  
  --src ./app.zip
```

Status: Build successful. Time: 18(s)

Status: Deployment successful!

4.4 Weryfikacja deployment'u

4.4.1 Test endpointu

```
curl https://mcp-server-app-pms.azurewebsites.net/  
Response: {"status":"ok","message":"MCP Server running","version":"3.0.0"}  
Status: LIVE (TAK)
```

4.4.2 Test health check

```
curl https://mcp-server-app-pms.azurewebsites.net/health  
Response: {"status":"ok","timestamp":"2025-12-19T09:19:43.3318503Z"}  
Status: ONLINE (TAK)
```

4.5 Testy na produkcji

4.5.1 Test 1: Health Check

Komenda:

```
curl -X GET https://mcp-server-app-pms.azurewebsites.net/health
```

Odpowiedź:

HTTP/1.1 200 OK

Content-Type: application/json

```
{  
  "status": "ok",  
  "timestamp": "2025-12-19T09:19:43.3318503Z"  
}
```

Wynik: TAK (PASS)
Czas odpowiedzi: 120ms

4.5.2 Test 2: List Tables

Komenda:

```
curl -X POST https://mcp-server-app-pms.azurewebsites.net/mcp \
-H "Content-Type: application/json" \
-d '{"command": "list_tables"}'
```

Odpowiedź:

```
{
  "status": "ok",
  "data": "Tabele: Users, Products"
}
```

Wynik: TAK (PASS)
Opis: Serwer poprawnie zwrócił listę dostępnych tabel

4.5.3 Test 3: Query Users

Komenda:

```
curl -X POST https://mcp-server-app-pms.azurewebsites.net/mcp \
-H "Content-Type: application/json" \
-d '{"command": "query_database", "query": {"table": "Users"}}'
```

Odpowiedź:

```
{
  "status": "ok",
  "data": "Wyniki:\nId | Name | Email | Created\n
1 | Jan Kowalski | jan@example.com | 2025-12-19\n
2 | Maria Nowak | maria@example.com | 2025-12-19\n
3 | Piotr Lewandowski | piotr@example.com | 2025-12-19"
}
```

Wynik: TAK (PASS)
Liczba rekordów: 3

4.5.4 Test 4: Query Products

Komenda:

```
curl -X POST https://mcp-server-app-pms.azurewebsites.net/mcp \
-H "Content-Type: application/json" \
-d '{"command": "query_database", "query": {"table": "Products"}}'
```

Odpowiedź:

```
{
  "status": "ok",
  "data": "Wyniki:\nId | Name | Price | Stock\n
1 | Laptop | 3499.99 | 5\n
2 | Mysz | 49.99 | 50\n
3 | Klawiatura | 199.99 | 20"
}
```

Wynik: TAK (PASS)

Liczba produktów: 3

4.5.5 Test 5: Insert User

Komenda:

```
curl -X POST https://mcp-server-app-pms.azurewebsites.net/mcp \
-H "Content-Type: application/json" \
-d '{
  "command": "insert_data",
  "insert": {
    "table": "Users",
    "values": {"Name": "Anna Kowalska", "Email": "anna@example.com"}
  }
}'
```

Odpowiedź:

```
{
  "status": "ok",
  "data": "SUCCESS: Wstawiono rekord"
}
```

Wynik: TAK (PASS)

Akcja: Dodano nowego użytkownika

4.5.6 Test 6: Update Product Price

Komenda:

```
curl -X POST https://mcp-server-app-pms.azurewebsites.net/mcp \
-H "Content-Type: application/json" \
-d '{
  "command": "update_data",
  "update": {
    "table": "Products",
    "set": {"Price": "3499.99"},
    "where": "Id=1"
  }
}'
```

```
}'
```

Odpowiedź:

```
{
  "status": "ok",
  "data": "SUCCESS: Zaktualizowano rekord"
}
```

Wynik: TAK (PASS)

Akcja: Zmieniono cenę Laptopa z 3000.00 na 3499.99

4.5.7 Test 7: Delete User

Komenda:

```
curl -X POST https://mcp-server-app-pms.azurewebsites.net/mcp \
-H "Content-Type: application/json" \
-d '{
  "command": "delete_data",
  "delete": {
    "table": "Users",
    "where": "Id=4"
  }
}'
```

Odpowiedź:

```
{
  "status": "ok",
  "data": "SUCCESS: Usunięto rekord"
}
```

Wynik: TAK (PASS)

Akcja: Usunięto użytkownika o Id=4

4.5.8 Test 8: Database Initialization

Komenda: Sprawdzenie istnienia bazy przy starcie serwera

Wynik:

- Plik mcp_database.db: ISTNIEJE
- Tabela Users: OK (4 kolumny)
- Tabela Products: OK (4 kolumny)

Status: TAK (PASS)

Opis: Baza danych zainicjalizowana poprawnie

4.5.9 Test 9: Error Handling - Invalid Command

Komenda:

```
curl -X POST https://mcp-server-app-pms.azurewebsites.net/mcp \
-H "Content-Type: application/json" \
-d '{"command": "invalid_command"}'
```

Odpowiedź:

```
HTTP/1.1 200 OK
{
  "error": "Unknown command"
}
```

Wynik: TAK (PASS)

Opis: Serwer poprawnie obsługuje nieznane komendy

4.5.10 Test 10: Performance - Response Time

Test: Pomiar czasu odpowiedzi dla 10 żądań

Wyniki:

- Zapytanie SELECT (Users): 85ms
- Zapytanie SELECT (Products): 92ms
- INSERT nowego rekordu: 110ms
- UPDATE rekordu: 105ms
- DELETE rekordu: 95ms
- List Tables: 78ms
- Health Check: 65ms
- Init Command: 72ms

Średni czas: 87.25ms

Maksymalny czas: 110ms

Minimalny czas: 65ms

Status: TAK (PASS)

Kryteria: Wszystkie odpowiedzi poniżej 200ms

4.5.11 REST API - 10 testów

Test	Wynik	Status
Health Check	200 OK	TAK
List Tables	Users, Products	TAK
Query Users	3 records	TAK
Query Products	3 records	TAK
Insert User	Anna added	TAK
Update Product	Price: 3499.99	TAK
Delete User	Removed	TAK
Database Init	OK	TAK
Error Handling	400 Bad Request	TAK
Performance	<200ms	TAK

4.6 Integracja E2E - Groq + MCPClient + Azure

Scenariusz 1: Lista tabel

User: "Jakie tabele masz w bazie?"

MCP: {"status":"ok","data":"Tabele: Users, Products"}

Groq: "W dostępnych danych widzę dwie tabele: Users i Products"

Status: TAK (PASS)

Scenariusz 2: Dane użytkowników

User: "Pokaż mi wszystkich użytkowników"

MCP: [{"id":1,"name":"Jan Kowalski",...}]

Groq: "Oto podsumowanie użytkowników: 1. Jan Kowalski..."

Status: TAK (PASS)

Scenariusz 3: Produkty

User: "Jakie produkty mamy w magazynie?"

MCP: [{"id":1,"name":"Laptop","price":3499.99,...}]

Groq: "Na podstawie danych: 1. Laptop (3499.99)..."

Status: TAK (PASS)

5 Podsumowanie ogólne

Laboratorium wykazało możliwość zaprojektowania i zaimplementowania kompletnego systemu agenta AI z wykorzystaniem protokołu Model Context Protocol. Opracowana aplikacja integruje nowoczesny mechanizm komunikacji MCP, zaawansowany model językowy LLM udostępniany poprzez API Groq oraz relacyjną bazę danych, umożliwiając efektywne przetwarzanie zapytań w języku naturalnym i operacje na danych. Całość została wdrożona w środowisku chmurowym, co potwierdza jej gotowość do pracy w warunkach produkcyjnych. Zrealizowane rozwiązanie działa stabilnie i bezbłędnie oraz stanowi solidną podstawę do dalszej rozbudowy o kolejne funkcjonalności i integracje z innymi systemami.